Monolithic vs Microservices 🧩

This is an important **KCNA / Cloud Native** concept. Let's understand it in very simple terms.

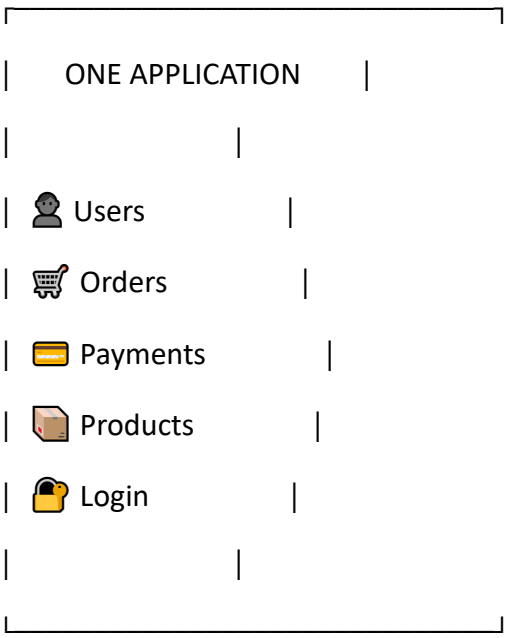
1. Monolithic Architecture

Simple definition

Monolithic = The whole application is built as one single unit.

Imagine a restaurant where **one kitchen handles everything**:

🍽️ MONOLITHIC APPLICATION



Everything is inside one application.

Example

An online shopping application may contain:

Monolithic App

|

├── Login

├── Products

├── Shopping Cart

├── Orders

├── Payments

└── Users

All of these are part of **one application**.

---

2. Problem with Monolithic Applications

Suppose you only want to update the **Payment** part.

In a monolithic application:

Payment

↓

Part of the big application

↓

Deploy whole application

You may need to build and deploy the **entire application**.

Also, if one major part fails, it can affect the whole application.

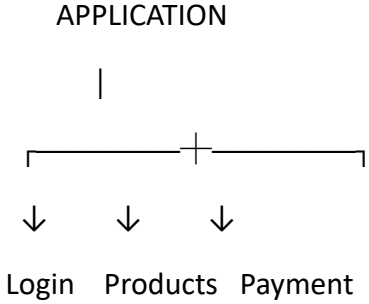
---

3. Microservices Architecture

Simple definition

Microservices = An application is divided into small, independent services.

Instead of one big application:



Service Service Service

Each service performs a specific job.

🛒 **Example: Online Shopping**

Instead of one big application:

Microservices

|

├── 👤 User Service

├── 📦 Product Service

├── 🛒 Order Service

├── 💳 Payment Service

└── 🔐 Authentication Service

Each service can be developed and deployed separately.

---

4. Real-Life Example 🏢

Monolithic

Think of **one large building**:

🏢 ONE BIG BUILDING

Everything is inside:

├── Bank

├── Restaurant

├── Shop

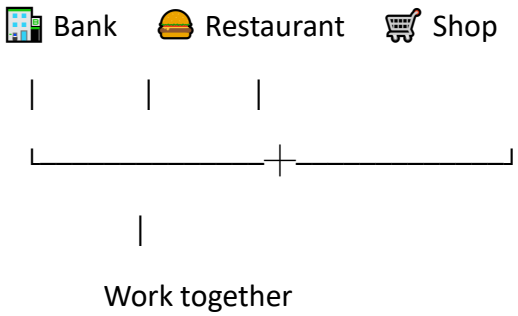
├── Office

└── Storage

If there is a major problem with the building, many things can be affected.

Microservices

Think of **separate small buildings**:



Each one can operate independently.

---

5. Main Differences ⭐

Monolithic	**Microservices**
One large application	Many small services
Components are tightly connected	Services are more independent
Usually deployed as one unit	Services can be deployed independently
Scaling can involve the whole application	Individual services can be scaled
Easier to start	More complex to manage
A large codebase	Smaller codebases
One technology stack is common	Different technologies can be used
Failure can affect a larger part of the application	Failure can potentially be isolated to one service

---

6. Scaling Example ⭐

Suppose your shopping website has:

100 users → Product Service

10,000 users → Payment Service

With microservices, you can scale **only the Payment Service**:

Payment Service

↓

┌─────┬─────┬─────┐

| Pod | Pod | Pod |

└─────┴─────┴─────┘

You don't necessarily need to scale the entire application.

This is one reason microservices are useful in cloud-native environments.

---

7. Why Microservices are Important for Kubernetes?

Kubernetes is very suitable for managing **many containerized services**.

For example:

Kubernetes Cluster

|

├── User Service

| └── Pods

|

├── Product Service

| └── Pods

|

├── Order Service

| └── Pods

|

└── Payment Service

└── Pods

Kubernetes can help with:

- Deploying services
- Scaling services
- Restarting failed containers
- Networking between services
- Rolling updates
- Service discovery

🧠 **KCNA Revision Notes**

Monolithic

One big application containing all components.

ONE BIG APP

┌──────────────────────┐

| Login |

| Products |

| Orders |

| Payments |

└──────────────────────┘

Microservices

Application divided into small, independent services.

Login — Product — Order — Payment

| | | |

Service Service Service Service

⭐ **Easy memory trick**

Monolithic = ONE BIG BLOCK 🧱

Microservices = MANY SMALL BLOCKS 🧱🧱🧱

And remember:

Microservices are not the same thing as containers. Containers are a way to package/run applications; microservices are an **architecture/design approach**. A microservice is often packaged in a container and managed by Kubernetes